# Camera calibration with a planar chessboard

Robot Perception 2026/2 · Exercise report · Aline Timm · PhD in Computational Modeling, FURG · October 2026

## 1 Objective

The exercise asks for the intrinsic parameters of a camera of choice, estimated from about twenty images of a printed chessboard, together with the distortion coefficients, the reprojection error and distortion-corrected images. The camera chosen was the main rear camera of an iPhone 13, since the same camera is a candidate sensor for the bench experiments of the course project. The report also documents the failed and partial attempts, because they show which properties of the image set make the parameters observable.

## 2 Setup

**Camera.** iPhone 13, rear wide module (EXIF: 5.1 mm, f/1.6, 26 mm equivalent), photos taken in portrait orientation at 3024×4032 px in HEIC and converted to JPEG (quality 95) after applying the EXIF orientation. The 35 mm equivalent focal length gives an independent estimate of the focal length in pixels, $f \approx 26 / 43.27 \times \sqrt{(3024^2 + 4032^2)} \approx 3\,028$ px, where 43.27 mm is the diagonal of the 35 mm frame. This value is used only to check the result and never enters the estimation.

**Target.** calib.io pattern with 8×11 squares of 15 mm (10×7 inner corners), printed on A4 paper. The sheet was not mounted on a rigid backing, a choice whose consequences appear in Section 5.

**Image set.** 28 images in three groups, acquired in sequence as the results of each group were analysed. Group A (10 images) has the sheet flat on a table or on a chair seat, photographed from above. Group B (10) repeats this geometry with small changes in camera angle. Group C (8) has the camera low and close to the sheet, so the board appears strongly foreshortened. Figure 1 shows all of them. The board tilt angles in Table 2 come from the final calibration.

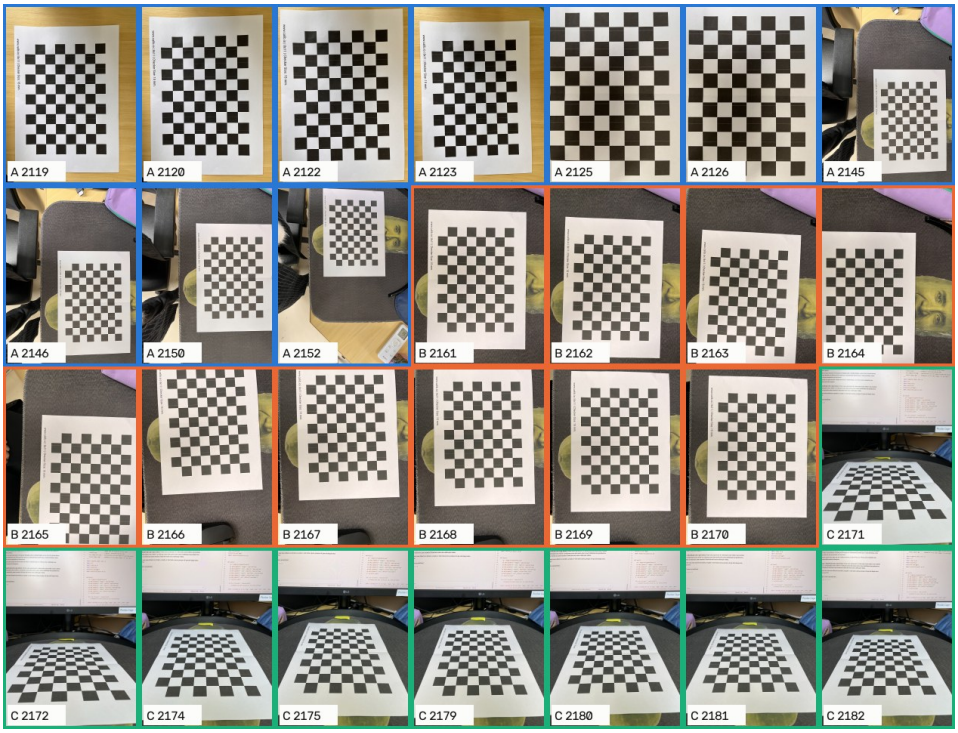



**Figure 1.** The 28 input images. Frame color marks the group: blue A (top-down, flat sheet), orange B (top-down, small tilt), green C (oblique). Labels are the last digits of the file names.

## 3 Method

Calibration follows Zhang's method as implemented in OpenCV 5.0 (*cv2.calibrateCamera*). The board defines a world frame with Z = 0, so each detected corner $p_i$ has a known 3D point $P_i$, and

the parameters are the ones that minimise the reprojection error over all images, as formulated in the lecture,

$$\min_{K,\,R,\,T,\,k} \Sigma_i \,\| \, p_i - \pi(K, k, R, T, P_i) \, \|^2$$

where π projects with the pinhole model and applies the distortion model. Two distortion models were fitted. The radial model of the lecture uses $x = x_d(1 + k_1 r^2 + k_2 r^4)$, and the default OpenCV model adds $k_3$ and two tangential terms $p_1$, $p_2$. Corners were detected with *findChessboardCornersSB* on a copy downscaled to 1600 px, then refined with *cornerSubPix* at full resolution. Without the downscaling the classic detector found the board in only 9 of the first 20 phone images. The code is in Appendix A; it writes K, the distortion vector, the extrinsics and the per-image error to a JSON file and produces the figures of the distortion-corrected images.

## 4 Results

With all 28 images the board was detected in every image and the radial model converged to $f_x$ = 3 070.9 px and $f_y$ = 3 070.0 px, within 1.4% of the EXIF estimate, with an aspect ratio of 1.0003. The principal point lies 3 px from the horizontal image center and 58 px from the vertical one. The overall RMS reprojection error is 2.50 px, about 0.08% of the image width. Table 1 lists the parameters of both models.

**Table 1.** Intrinsic parameters from the 28 images. Image size 3024×4032 px.

| Parameter | Radial model ($k_1$, $k_2$) | OpenCV model (5 terms) |
|---|---|---|
| $f_x$, $f_y$ (px) | 3 070.9, 3 070.0 | 3 042.8, 3 042.7 |
| $c_x$, $c_y$ (px) | 1 515.1, 1 958.5 | 1 523.2, 2 036.0 |
| $k_1$, $k_2$ | 0.0607, -0.1583 | 0.1247, -0.5818 |
| $p_1$, $p_2$, $k_3$ | fixed at 0 | 0.0011, 0.0011, 0.7613 |
| RMS reprojection error (px) | 2.501 | 2.454 |

The camera matrix of the radial model, used in the rest of the report, is

$$K = \begin{vmatrix} 3\,070.9 & 0 & 1\,515.1 \\ 0 & 3\,070.0 & 1\,958.5 \\ 0 & 0 & 1 \end{vmatrix} \text{ (px)}$$

**Table 2.** Radial-model calibration for each combination of groups. Tilt is the angle between the board normal and the optical axis, from the final calibration.

| Images | n | Board tilt (deg) | $f_x$ (px) | $f_x/f_y$ | RMS (px) |
|---|---|---|---|---|---|
| A | 10 | 0 to 5 | 5 840.4 | 1.000 | 1.76 |
| A+B | 20 | 0 to 5 | 6 429.8 | 1.001 | 1.83 |
| C | 8 | 60 to 78 | 3 295.1 | 1.505 | 2.99 |
| A+B+C | 28 | 0 to 78 | 3 070.9 | 1.000 | 2.50 |
| EXIF estimate | | | 3 028 | 1 | |

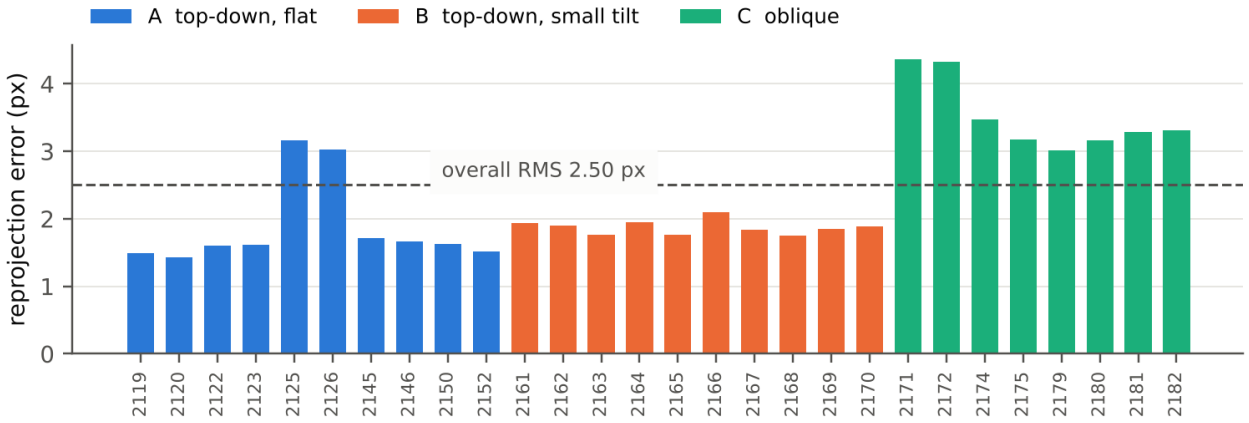



**Figure 2.** RMS reprojection error of each image in the final calibration (radial model). Colors follow Figure 1.

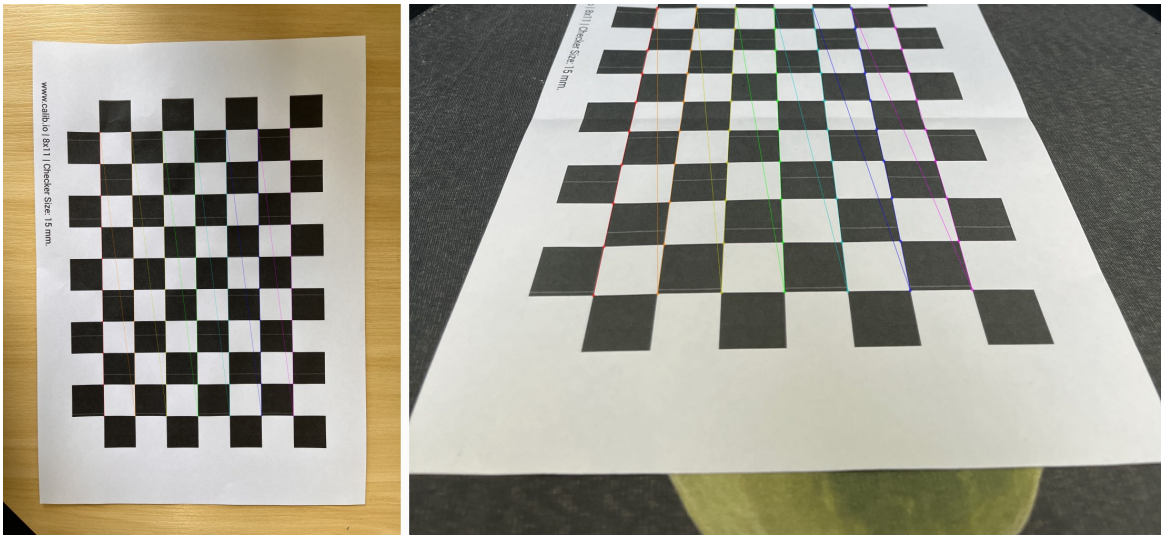


**Figure 3.** Detected corners in a top-down image (2120, group A) and an oblique one (2179, group C, cropped to the board). The colored polyline gives the corner order used to match the 3D board points.

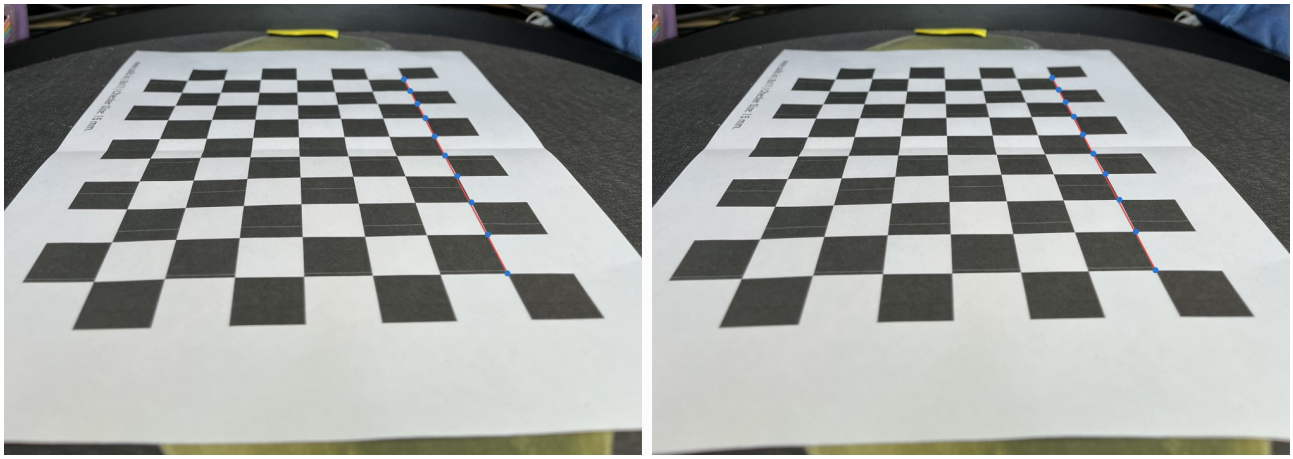


**Figure 4.** Image 2171 before (left) and after (right) distortion correction with the radial model, cropped to the board. The blue points are one column of detected corners and the red line joins its end points. The column deviates at most 8.1 px from that line before correction and 7.9 px after.

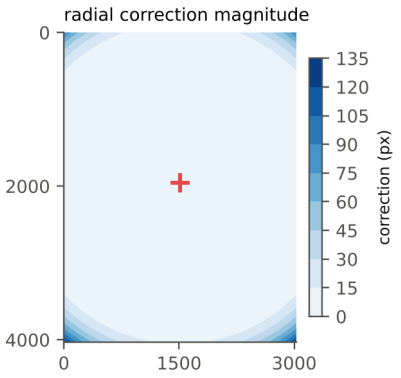



**Figure 5.** Magnitude of the radial correction over the image, with the principal point marked. The correction is below 5 px on 40% of the image and below 15 px on 93% of it (median 6 px). Values above 100 px appear only in the extreme corners, a region no image of the set covers with board corners, so the model there is an extrapolation.

## 5 Discussion

### 5.1 A low reprojection error did not mean a correct calibration

The first phone calibration used only the top-down images. It reached an RMS of 1.83 px with 20 images, lower than the final value, and still placed the focal length at 6 430 px, more than twice the EXIF estimate. Figure 6a explains why. Holding f fixed at values between 2400 and 8000 px and re-optimising everything else, the cost of the top-down set changes by less than 0.05 px. For a board parallel to the image plane every corner has the same depth Z, the projection $u = f\,X / Z + c_x$ depends only on the ratio f / Z, and a larger focal length is matched exactly by a more distant board. The optimiser picks a point along this valley and the distortion terms absorb the small residual, which is why $k_2$ reached −2.6 in that run. Once the oblique images enter, depth varies across the board, the ratio splits into its two factors and the cost gains a clear minimum near 3 071 px.

Group C on its own failed in a different way, with $f_x/f_y = 1.51$. All eight oblique images were taken from the same side of the sheet, so the board is rotated about a single axis and only one focal direction is constrained. Plane-based calibration is singular when all board planes are parallel to a common direction (Sturm and Maybank, 1999); the combination of the three groups supplies that, and the aspect ratio returns to 1.000.

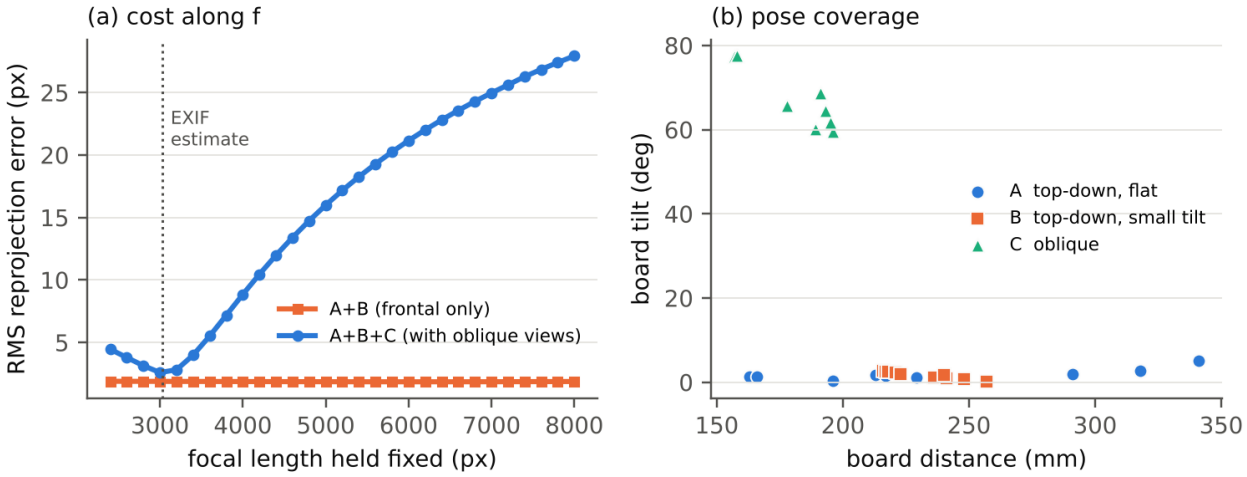



**Figure 6.** (a) RMS error with the focal length held fixed, for the top-down images (A+B) and for the full set (A+B+C). (b) Distance and tilt of the board in each image, from the final calibration.

### 5.2 Where the remaining error comes from

The oblique images carry the largest errors, between 3.0 and 4.4 px, while most top-down ones stay between 1.4 and 2.1 px (Figure 2). Lens distortion does not account for the difference. A

straight line of corners on a flat board must remain straight under any perspective, and in Figure 4 the corner column bends by 8.1 px before correction and by 7.9 px after it. The sheet itself is not flat. It shows a visible fold line across the middle, and in group C it rested on a cushioned chair seat. At tilts of 60 to 77 degrees a small bend of the paper moves the corners along the direction where the image is most compressed, and the near edge of the board is also out of focus at that distance. Images 2125 and 2126, the closest ones in group A, show the same effect at a smaller scale.

### 5.3 Distortion and model choice

The distortion found is small ($k_1$ = 0.061, $k_2$ = -0.158). This is consistent with the iPhone image pipeline applying its own geometric lens correction before the photo is saved, which leaves only a residual to be modelled here. The five-parameter OpenCV model lowers the RMS by only 0.05 px and does so with $k_2$ and $k_3$ of opposite sign (-0.58 and 0.76), a pattern of correlated terms compensating each other rather than describing the lens. The two-term radial model of the lecture is the better-conditioned choice for this camera, and it is the one adopted.

### 5.4 Earlier attempts with a laptop webcam

Before the phone, two sets of eight images were taken with a 1280×960 laptop webcam, holding the sheet in front of the camera. In the first set the board was detected in five images; the sheet bent in the hand, the views were frontal, and the result had an RMS of 3.41 px with $f_x$ = 2090 px, $f_y$ = 1907 px and a principal point 273 px below the center. The second set reduced the RMS to 1.65 px, yet its focal length (1726 px) moved to 1538 px when both sets were combined. Both failures repeat the two causes discussed above, a non-planar target and views without enough tilt, and motivated the change of camera and of acquisition procedure.

## 6 Conclusion

The final calibration of the iPhone 13 wide camera gives f ≈ 3 071 px, square pixels, a principal point close to the image center and an RMS error of 2.50 px, with the focal length confirmed by the value derived from EXIF. Reaching it required changing how the images were acquired more than how they were processed. The reprojection error alone hid a wrong focal length for most of the process, and the check against an independent estimate together with the cost sweep of Figure 6a exposed it. A new acquisition should mount the sheet on a rigid flat board, tilt the camera about both axes from several sides, and place the board near the image corners so that the distortion model is supported by data where its correction is largest.

## References


Z. Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330-1334, 2000.

P. Sturm and S. Maybank. On plane-based camera calibration: a general algorithm, singularities, applications. *Proc. IEEE CVPR*, 1999.

G. Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000.


## Appendix A Calibration code (calibrate.py)

Run as *python calibrate.py --images fotos/ --cols 10 --rows 7 --square 15 --simple*. The analysis behind Figures 1 to 6 (subset calibrations, focal-length sweep, distortion map) is in a separate script, analysis.py, built on the same functions.

```
"""
Camera calibration with a chessboard (Zhang's method, OpenCV).

Usage:
    python calibrate.py --images fotos/ --cols 9 --rows 6 --square 25

    --cols / --rows : number of INNER corners (squares - 1 in each direction)
    --square        : side of one square in mm (measure the printed board with a ruler)
    --simple        : radial model only (k1, k2), as in the lecture. Without it,
                      OpenCV's full model is used (k1, k2, p1, p2, k3).
Outputs (in ./resultados):
```

```
    params.json             K, distortion, RMS error, per-image error
    corners/                images with the detected corners drawn
    undistorted/            original | corrected side by side
    per_image_error.png     reprojection error per image (to spot bad photos)
    mosaic_inputs.jpg       thumbnails of all input images, for the report
"""
import argparse, glob, json, os
import cv2
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--images", required=True)
    ap.add_argument("--cols", type=int, required=True)
    ap.add_argument("--rows", type=int, required=True)
    ap.add_argument("--square", type=float, required=True)
    ap.add_argument("--simple", action="store_true")
    ap.add_argument("--out", default="resultados")
    a = ap.parse_args()

    for d in ("corners", "undistorted"):
        os.makedirs(os.path.join(a.out, d), exist_ok=True)

    paths = sorted(p for ext in ("jpg", "jpeg", "png", "JPG", "JPEG", "PNG")
                   for p in glob.glob(os.path.join(a.images, f"*.{ext}")))
    if not paths:
        raise SystemExit(f"No images found in {a.images}")

    # 3D points of the board in its own frame (Z = 0), in mm
    pattern = (a.cols, a.rows)
    objp = np.zeros((a.rows * a.cols, 3), np.float32)
    objp[:, :2] = np.mgrid[0:a.cols, 0:a.rows].T.reshape(-1, 2) * a.square

    obj_pts, img_pts, used, size = [], [], [], None
    crit = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 1e-3)

    for p in paths:
        img = cv2.imread(p)
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        if size is None:
            size = gray.shape[::-1]
        elif gray.shape[::-1] != size:
            print(f"[skip] {os.path.basename(p)}: different resolution {gray.shape[::-1]}")
            continue
        # detect on a downscaled copy (fast and more robust on 12 MP phone photos),
        # then refine the corners at full resolution
        scale = min(1.0, 1600.0 / max(gray.shape))
        small = cv2.resize(gray, None, fx=scale, fy=scale, interpolation=cv2.INTER_AREA) if scale < 1 else gray
        ok, corners = cv2.findChessboardCornersSB(
            small, pattern, cv2.CALIB_CB_NORMALIZE_IMAGE + cv2.CALIB_CB_EXHAUSTIVE + cv2.CALIB_CB_ACCURACY)
        if not ok:
            ok, corners = cv2.findChessboardCorners(
                small, pattern, cv2.CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_NORMALIZE_IMAGE)
        if not ok:
            print(f"[skip] {os.path.basename(p)}: board not found")
            continue
        corners = (corners.reshape(-1, 1, 2) / scale).astype(np.float32)
        win = max(5, int(11 / scale) // 2)
        corners = cv2.cornerSubPix(gray, corners, (win, win), (-1, -1), crit)
        obj_pts.append(objp)
        img_pts.append(corners)
        used.append(p)
        vis = cv2.drawChessboardCorners(img.copy(), pattern, corners, ok)
        cv2.imwrite(os.path.join(a.out, "corners", os.path.basename(p)), vis)

    print(f"\nBoard detected in {len(used)} of {len(paths)} images")
    if len(used) < 3:
        raise SystemExit("Too few valid images (aim for 15 to 20).")

    flags = cv2.CALIB_FIX_K3 | cv2.CALIB_ZERO_TANGENT_DIST if a.simple else 0
    rms, K, dist, rvecs, tvecs = cv2.calibrateCamera(obj_pts, img_pts, size, None, None, flags=flags)

    # per-image RMS reprojection error
    per_img = []
    for op, ip, r, t in zip(obj_pts, img_pts, rvecs, tvecs):
        proj, _ = cv2.projectPoints(op, r, t, K, dist)
        diff = ip.reshape(-1, 2) - proj.reshape(-1, 2)
        per_img.append(float(np.sqrt(np.mean(np.sum(diff ** 2, axis=1)))))

    np.set_printoptions(precision=4, suppress=True)
    print(f"\nImage size: {size[0]} x {size[1]}")
    print(f"RMS reprojection error: {rms:.4f} px")
    print("K =\n", K)
    print("dist (k1, k2, p1, p2, k3) =", dist.ravel())
    print(f"fx/fy = {K[0,0]/K[1,1]:.4f}   principal point offset from center: "
```

```
          f"({K[0,2]-size[0]/2:.1f}, {K[1,2]-size[1]/2:.1f}) px")
    worst = np.argsort(per_img)[::-1][:3]
    print("Worst images:", [(os.path.basename(used[i]), round(per_img[i], 3)) for i in worst])

    with open(os.path.join(a.out, "params.json"), "w") as f:
        json.dump({
            "image_size": size, "model": "k1,k2 only" if a.simple else "k1,k2,p1,p2,k3",
            "rms_px": rms, "K": K.tolist(), "dist": dist.ravel().tolist(),
            "per_image_error_px": dict(zip(map(os.path.basename, used), per_img)),
            "extrinsics": [{"image": os.path.basename(p), "rvec": r.ravel().tolist(),
                            "tvec_mm": t.ravel().tolist()} for p, r, t in zip(used, rvecs, tvecs)],
        }, f, indent=2)

    # undistortion, original | corrected
    newK, roi = cv2.getOptimalNewCameraMatrix(K, dist, size, 1, size)
    for p in used:
        img = cv2.imread(p)
        und = cv2.undistort(img, K, dist, None, newK)
        both = np.hstack([img, und])
        cv2.imwrite(os.path.join(a.out, "undistorted", os.path.basename(p)), both)

    # per-image error chart
    names = [os.path.basename(p) for p in used]
    plt.figure(figsize=(max(6, len(names) * 0.45), 4))
    plt.bar(range(len(names)), per_img, color="#E8622C")
    plt.axhline(rms, color="#3B1E2A", ls="--", label=f"overall RMS = {rms:.3f} px")
    plt.xticks(range(len(names)), names, rotation=70, fontsize=7)
    plt.ylabel("reprojection error (px)")
    plt.legend()
    plt.tight_layout()
    plt.savefig(os.path.join(a.out, "per_image_error.png"), dpi=150)

    # mosaic of input images
    thumbs = [cv2.resize(cv2.imread(p), (240, int(240 * size[1] / size[0]))) for p in used]
    cols = 5
    while len(thumbs) % cols:
        thumbs.append(np.zeros_like(thumbs[0]))
    rows = [np.hstack(thumbs[i:i + cols]) for i in range(0, len(thumbs), cols)]
    cv2.imwrite(os.path.join(a.out, "mosaic_inputs.jpg"), np.vstack(rows))

    print(f"\nResults saved to {a.out}/")


if __name__ == "__main__":
    main()
```